

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I MUKKARAM (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MUKKARAM

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drone for Emergency Medicine Delivery (Greedy Best-First Search vs A*)

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path that appears closest to the destination using only the heuristic value $h(n)$ (straight-line distance).

Working:

Uses $f(n) = h(n)$.

Chooses the node with the smallest estimated distance to the goal.

Ignores the actual travel cost.

Strengths

Very fast.

Low memory usage.

Suitable when an immediate decision is required.

Weaknesses

May choose blocked or risky routes.

Does not guarantee the shortest or safest path.

Can get trapped in dead ends if heuristic is misleading.

ii. A* Search

A* combines actual travel cost and heuristic estimate.

Formula

$$f(n) = g(n) + h(n)$$

Where:

$g(n)$ = Actual cost from start to current node.

$h(n)$ = Estimated cost to goal.

Working

Considers both distance and travel cost.

Avoids expensive or dangerous routes.

Recalculates better paths when costs change.

Advantages

Finds optimal path (with admissible heuristic).

Balances speed and safety.

Better suited for emergency delivery.

iii. Impact of Heuristic Accuracy

Accurate Heuristic

GBFS quickly reaches goal.

A* expands fewer nodes and remains optimal.

Inaccurate Heuristic

GBFS may choose poor routes.

A* performance decreases but still finds the best solution if heuristic is admissible.

iv. Effect of Dynamic Changes

When roads become blocked:

GBFS may continue toward blocked paths and require restarting.

A* updates path costs and searches for alternative routes more effectively.

Frequent environmental changes increase computation for both algorithms.

v. Recommended Algorithm

Recommended: A*

Justification

Produces optimal and safe routes.

Considers terrain, weather, and travel cost.

Handles dynamic obstacles better.

Suitable for emergency medicine delivery where safety is more important than speed alone.

2. Smart City Traffic Signal Optimization (Hill Climbing & Genetic Algorithm)

Optimization Problem

Objective

Minimize waiting time.

Minimize fuel consumption.

Reduce traffic congestion.

Variables

Signal timing.

Green-light duration.

Traffic flow.

Traditional search is inefficient because:

Huge search space.

Dynamic traffic conditions.

Exponential number of possible signal combinations.

i. Hill Climbing

Hill Climbing starts with an initial signal plan and repeatedly moves to a better neighboring solution.

Advantages

Fast.

Easy to implement.

Uses little memory.

Limitations

May stop at local optimum.

Cannot guarantee global optimum.

Sensitive to starting solution.

ii. Local Maxima, Plateaus and Ridges

Local Maximum

AI finds a good solution but misses a better one.

Example: Traffic improves in one area but overall congestion remains.

Plateau

Many neighboring solutions have equal quality.

Example: Changing signal timing has almost no effect.

Ridge

Best solution requires moving in multiple directions.

Example: Several intersections must be adjusted together.

iii. Simulated Annealing / Genetic Algorithm

Simulated Annealing

Occasionally accepts worse solutions.

Escapes local maxima.

Gradually converges.

Genetic Algorithm

Population of solutions.

Uses selection, crossover, mutation.

Explores many regions simultaneously.

Finds high-quality global solutions.

iv. Recommendation

Recommended: Genetic Algorithm

Reason

Handles large search spaces.

Adapts to changing traffic.

Avoids local optima.

Highly scalable for smart cities.

3. Mars Rover (Online Search Agent)

i. Why Online Search?

The rover has:

Unknown terrain.

No complete map.

Limited sensing.

Real-time decision making.

Offline search requires complete information, which is unavailable.

Hence Online Search is required.

ii. Challenges

Partial Observability

The rover cannot see the entire environment.

Dynamic Environment

Dust storms, rocks, and terrain may change.

Limited Resources

Battery and computation are limited.

Safety

Must avoid hazards while exploring.

iii. Internal Model

The rover continuously:

Stores visited locations.

Marks obstacles.

Updates terrain map.

Learns safe paths.

Improves future navigation.

iv. Online vs Uninformed vs Informed Search

Uninformed Search

No heuristic.

Explores blindly.

Slow.

Informed Search

Uses heuristic.

Faster when map is known.

Online Search

Learns while moving.

Best for unknown environments.

Continuously updates knowledge.

v. Recommendation

Recommended: Online Search Agent

Reason

Works with incomplete information.

Adapts to new obstacles.

Safe and flexible.

Suitable for Mars exploration.

4. Exam Timetable using Constraint Satisfaction Problem (CSP)

CSP Formulation

Variables

Exams

Domains

Available time slots

Available exam halls

Constraints

No student has overlapping exams.

Hall capacity must not exceed limit.

Some exams occur before others.

Faculty must be available.

Special accommodation students receive suitable rooms.

i. Variables, Domains and Constraints

Variables represent exams.

Domains represent available slots and halls.

Constraints restrict invalid schedules.

The goal is to satisfy all constraints simultaneously.

ii. Backtracking Search

Steps:

Assign a time slot.

Check constraints.

If conflict occurs, undo assignment.

Try another slot.

Continue until all exams are scheduled.

iii. Constraint Propagation (Forward Checking)

Forward checking removes invalid future choices after every assignment.

Benefits:

Detects conflicts early.

Reduces search space.

Speeds up scheduling.

Prevents unnecessary backtracking.

iv. Real-World Challenges

Thousands of students.

Limited halls.

Faculty changes.

Last-minute modifications.

Large search space.

Best Strategy Backtracking + Forward Checking + Constraint Propagation

Reason: Efficient, scalable, and reduces computation.

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Minimax assumes:

AI is the maximizing player.

Human is the minimizing player.

The algorithm explores possible moves and chooses the move with the highest guaranteed score.

ii. Computational Challenges

Large game trees create:

Millions of possible states.

High memory usage.

Long computation time.

Real-time constraints.

Time complexity: $O(b^d)$

where:

b = branching factor

d = search depth

iii. Alpha-Beta Pruning

Alpha-Beta removes branches that cannot influence the final decision.

Alpha (α)

Best value for maximizing player.

Beta (β)

Best value for minimizing player.

If $\alpha \geq \beta$, remaining branches are pruned.

Advantages

Explores fewer nodes.

Faster decision making.

Produces the same optimal move as Minimax.

iv. Evaluation Function

The evaluation function scores a game state.

Factors:

Number of units.

Health of units.

Resources collected.

Territory controlled.

Defensive strength.

Distance to enemy.

Strategic positions.

Higher score = Better position.

v. Recommendation

Recommended Strategy: Alpha-Beta Pruning with Minimax

Justification

Maintains optimal decision quality.

Greatly reduces computation.

Meets strict real-time deadlines.

Suitable for complex strategy games with very large search trees.

Quick Exam Tips

Greedy Best-First Search: $f(n) = h(n)$

A* Search: $f(n) = g(n) + h(n)$

Hill Climbing: Fast but can get stuck in local maxima.

Genetic Algorithm: Best for large optimization problems.

Online Search: Best for unknown environments.

CSP: Variables + Domains + Constraints.

Minimax: Optimal game decision-making.

Alpha-Beta Pruning: Same result as Minimax with much less computation.